Agent Wallet

Technical Documentation

Topic F

2026 FinTechathon

2026 Shenzhen International FinTech Competition (FinTechathon)
Xilihu FinTech College Students Challenge

Date	**:**	**15/10/2026**
Version	**:**	**1.0**
Status	**:**	**Completed**

Contents

Introduction....3
1 System architecture....3
1.1 Data flow and boundaries....3
1.2 Architecture Diagram....4
2 Core algorithm description....5
2.1 Interpret and activate a policy....5
2.2 Check eligibility before inference....5
2.3 Generate a typed proposal....5
2.4 Calculate the reference rebalance....5
2.5 Evaluate and route....5
2.6 Submit and reconcile....5
3 Security design....6
3.1 Audit and emergency controls....6
3.2 Security assumptions and limits....6
3.3 Verification and source references....6
3.4 Security Diagram....7

Introduction

This document explains how the policy-controlled blockchain agent is built, how it turns a user goal into a proposed action, and how execution is restricted. It gives developers, reviewers and project teammates a concise reference for understanding, maintaining and assessing the system.

The application demonstrates goal-based treasury automation using fake assets on local Anvil. AI recommends actions; independent policy, simulation, risk and smart account controls govern execution. The prototype is educational and does not manage real funds.

Read the architecture section for component responsibilities, the algorithm section for the processing flow, and the security section for authorization boundaries and limitations.

1 System architecture

The application is a modular monolith: one FastAPI backend, a React interface, PostgreSQL and a local blockchain. This keeps deployment simple while preserving separate responsibilities.

Component	Responsibility
React interface	Create/review policies; run or schedule checks; review proposals, approvals and audit history.
FastAPI and policy lifecycle	Authenticate the owner, validate inputs and manage immutable approved policy versions.
AI agent and provider adapter	Send an address free portfolio/policy snapshot to Ollama or optional OpenAI; validate HOLD or SWAP output.
Policy, simulation and risk	Apply deterministic rules, simulate the operation, check allocation and assign risk.
Execution and automation	Coordinate approvals, wallet reservations, rechecks, submission, receipt recovery and scheduled work.
Persistence and audit	Store versions, runs, proposals, simulation/risk records, approvals, executions and events.
Anvil contracts	PolicySmartAccount enforces independent restrictions; MockDex exchanges TestUSDC and TestWETH.

1.1 Data flow and boundaries

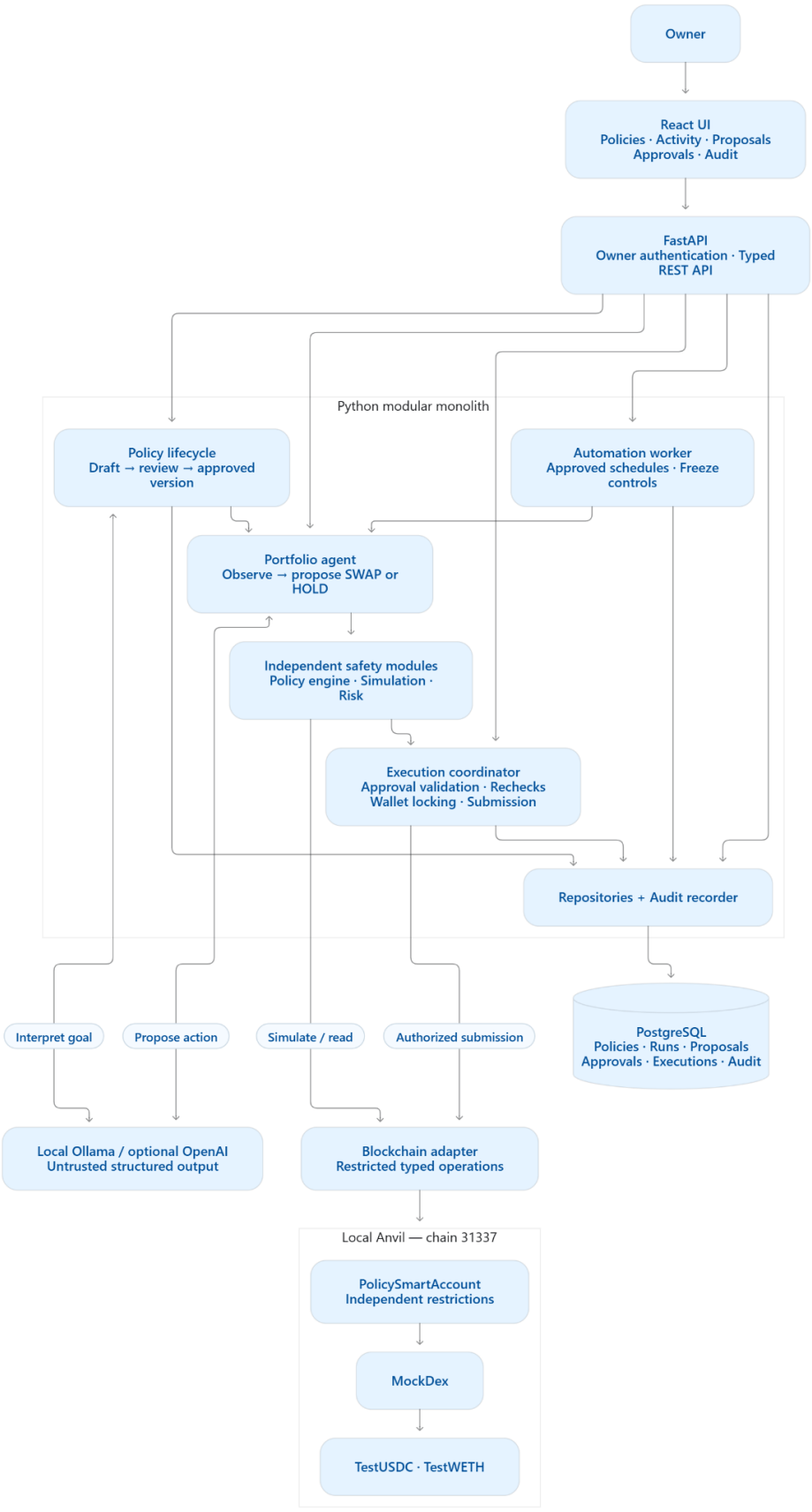
Owner goal → validated draft → explicit approval → wallet observation → AI proposal → independent checks → approval if needed → authorized submission → receipt and audit.

The agent cannot import execution authority. Policy rules and risk decisions do not depend on the LLM. Infrastructure free schemas define immutable boundary objects; repositories own database access. The simulation and smart account adapters isolate blockchain details.

A fresh deployment holds 500 TestUSDC and 500 TestWETH. Both use six decimals and a fixed 1:1 value. Policies use Decimal strings; token amounts use integer units. PostgreSQL is the normal store; isolated tests can use SQLite. Offline mode uses deterministic sizing.

Architecture reference: docs/architecture.md

1.2 Architecture Diagram



2 Core algorithm description

2.1 Interpret and activate a policy

PolicyInterpreter converts a natural language goal into a validated PolicyDraft. The owner reviews targets, budget, cadence, tolerance, slippage and any hard minimum. Approval activates a new immutable version. Replacing a policy stops its old schedule; scheduling requires separate owner enablement.

2.2 Check eligibility before inference

Manual and scheduled portfolio runs share readiness checks. Exhausted daily budget, unelapsed cadence, allocation within tolerance or a hard minimum that cannot be restored in one permitted trade prevents a model call. Pending work and freeze controls also prevent new execution. A durable generation reservation allows model calls outside wallet locks.

2.3 Generate a typed proposal

AITradingAgent supplies balances, remaining daily allowance, trade ceiling and approved policy to a StructuredProvider. The model returns HOLD or SWAP with assets, amount, slippage and reason. The backend preserves these trade fields and supplies trusted addresses, chain and operation. Invalid output fails without an executable fallback.

2.4 Calculate the reference rebalance

For equal price mock assets, let U be USDC units, W be WETH units, T = U + W, and n/d the exact USDC target fraction. The offline planner and readiness helper use:

distance = U × d − T × n
gap = floor(abs(distance) / d)
amount = min(gap, remaining daily allowance, per trade ceiling)

Positive distance means sell USDC; negative distance means sell WETH. Tolerance uses integer cross products. A hard minimum must hold after the permitted trade. This planner does not replace the live AI amount.

2.5 Evaluate and route

Policy rules check spending, assets, protocol, contract, identity, operation, cadence and slippage. RPC simulation uses eth_call and estimate_gas. Expected token deltas follow the fixed 1:1 mock exchange. Post trade allocation must strictly improve normalized target distance and satisfy any hard minimum.

Policy failure, simulation failure or HIGH risk means BLOCK. MEDIUM requires owner approval; otherwise a permitted LOW proposal may execute. A proposal consuming at least 70% of the approved daily limit is MEDIUM unless a HIGH finding dominates.

2.6 Submit and reconcile

The coordinator locks/reserves the wallet, revalidates the stored proposal and commits submission intent before RPC. It records the hash and reconciles the receipt. An uncertain send retains the reservation and requires investigation; it is never blindly retried.

Example: from 500/500 toward exactly 55% USDC, the gap is 50 WETH. Offline mode trades 50 with a 100 cap, or 30 then 20 with a 30 cap. Live AI may choose another permitted amount or HOLD.

Algorithm sources apps/api/app/agent/sizing.py

3 Security design

Security uses independent application and contract boundaries. AI output is untrusted even when it matches the JSON schema. Owner approval grants specific permissions rather than unrestricted model authority.

Boundary	Enforcement
Authentication and activation	Owner bearer token of at least 24 characters; constant time comparison. Only explicit owner approval activates a policy.
Typed operations	No model supplied executable addresses, keys or calldata. The application supports narrow stored swap proposals, not arbitrary raw transactions.
Independent gates	Deterministic policy, simulated allocation checks and risk findings govern permission. Invalid output, HIGH risk and failed simulation cannot execute.
Approval and concurrency	Ten minute approval binds proposal digest and active version. Fresh checks, wallet locks, durable reservations and unique execution records prevent stale/concurrent submission.
Smart account	Authorized agent, pause/revoke, daily spending, allowed targets/assets, minimum output and replay protection. Only owner changes restrictions; temporary allowances are cleared.
Local chain and recovery	Configuration and RPC guards require local Anvil 31337. Freeze stops scheduling and requests contract pause; resume leaves scheduling off. Unknown sends are not automatically repeated.

3.1 Audit and emergency controls

Audit links the approved version, AI proposal, rule results, simulation, risk, approval, timestamps, hash and final outcome. System skips and AI HOLD are distinct. Three consecutive scheduled failures/blocked checks freeze execution; repeated HOLD instead produces an advisory. Local freeze persists if the on chain pause request fails.

3.2 Security assumptions and limits

Contracts are unaudited and Anvil accounts are publicly known unlocked development identities. A compromised local RPC can impersonate the owner; a compromised backend can bypass off chain allocation and approval rules. The contract still enforces its narrower restrictions. Submitted transactions cannot be cancelled.

The app has no general text redactor, application level encryption, retention automation, KYC/AML or fraud screening. Owner tokens remain in browser memory; secrets use environment settings and sanitized errors. Audit is append only through the API, not tamper proof against administrators. Fixed prices and modeled deltas are unsuitable for real markets.

3.3 Verification and source references

Recorded cleanup evidence: 237 Python tests passed, six optional integrations skipped; four Anvil suites passed separately. Those dated results are not a fresh test run for this document. Critical tests cover overspending, invalid approvals, pause/revoke, replay and direct contract bypass.

Policy and simulation pipeline apps/api/app/execution/pipeline.py

Contract and independent controls contracts/src/PolicySmartAccount.sol

Threats and residual risks docs/threat-model.md

Dated verification docs/history/README.md

3.4 Security Diagram

